ExperimentIQ

Upgraded A/B Testing, Causal ML & Business Experiment Intelligence Platform

Complete project blueprint + skills-to-learn roadmap

Area	Plan
Project domain	Product analytics, business experimentation, causal machine learning
Core outcome	Analyze experiments, predict outcomes, estimate uplift, quantify business impact
Major additions	CUPED, SRM diagnostics, power/sample-size planning, T-/X-Learner uplift modeling
Product interface	Streamlit dashboard; optional FastAPI API
Suggested build style	Build a reliable statistical core first, then ML, causal ML, and deployment

Guiding principle: build a trustworthy experiment decision-support product, not just a collection of models. Statistical validity, leakage prevention, transparent uncertainty, and reproducibility matter more than the number of algorithms.

Prepared as an upgraded version of the existing ExperimentIQ blueprint, incorporating the four requested additions.

1. Executive Summary

ExperimentIQ helps a product or business team compare Variant A with Variant B. It combines statistical analysis, predictive ML, deep learning, uplift modeling, revenue impact estimation, and an interactive dashboard. It should recommend whether to deploy B, keep A, continue collecting data, or investigate data quality.

What the finished project should answer

- Did B outperform A, and by how much?
- How uncertain is the estimated effect? Are the sample and assignment quality acceptable?
- Can pre-experiment information improve statistical precision through CUPED?
- Which segments may benefit more or less from B, based on uplift estimates?
- What are the estimated incremental conversions, revenue, cost, and ROI?
- What is the next action, and what evidence supports it?

The upgraded feature set

Module	Purpose	Priority
Experiment validation	Data checks, leakage checks, sample ratio mismatch (SRM)	Must-have
Statistical engine	Conversion/revenue effects, confidence intervals, hypothesis tests	Must-have
Power planning	Minimum detectable effect, power and required sample size	Must-have
CUPED	Use pre-experiment covariates to reduce outcome variance	Advanced core
Predictive ML	Logistic Regression, Random Forest, XGBoost and calibration	Must-have
Deep learning	MLP benchmark for tabular conversion prediction	Stretch/core extension
Uplift / causal ML	T-Learner first; X-Learner as an extension	Advanced differentiator
Business impact	Incremental conversions, revenue, cost and ROI	Must-have
Dashboard and deployment	Streamlit app, hosted demo, README; optional API	Must-have

2. Skills to Learn — In the Right Order

Follow this sequence so each skill is used in the project. You do not need to master every topic before starting; learn a topic, implement it, then move on.

Stage 1 — Python foundations

- Python syntax, functions, modules, exceptions, file paths, virtual environments and package installation.
- Core data structures: lists, dictionaries, sets, tuples; comprehensions; type hints at a basic level.
- NumPy arrays, vectorized operations, broadcasting and random-number generation.
- Pandas DataFrames, joins, groupby, missing values, duplicate checks, datetime handling and CSV/Parquet I/O.
- Matplotlib or Plotly for clear charts; learn one first rather than switching between many libraries.

Use in project: load, clean, validate and summarize experiment data.

Stage 2 — SQL and data handling

- SQL SELECT, WHERE, GROUP BY, HAVING, JOIN, CASE, CTEs and window functions.
- Basic database concepts: primary keys, indexes, schemas and transactions.
- SQLite for a simple local prototype; PostgreSQL if you want persistent multi-user or hosted storage.
- Data validation rules: allowed categories, ranges, uniqueness, missingness and schema checks.

Use in project: store experiment records, query segment summaries and reproduce analyses.

Stage 3 — Statistics and experimental design

- Probability, distributions, expectation, variance, standard error and sampling.
- Hypothesis testing, null/alternative hypotheses, p-values, confidence intervals and effect sizes.
- Two-proportion tests / chi-square tests for binary outcomes; t-tests and suitable robust alternatives for continuous outcomes.
- Multiple comparisons, practical vs statistical significance, randomization, guardrail metrics and experiment stopping discipline.
- Power, significance level, minimum detectable effect (MDE), allocation ratio and sample-size calculation.
- Bootstrap confidence intervals and robust analysis for skewed revenue.

Use in project: implement a trustworthy A/B analysis before training ML models.

Stage 4 — Experiment diagnostics and CUPED

- Sample Ratio Mismatch (SRM): expected vs observed allocation and chi-square goodness-of-fit test.
- CUPED: pre-experiment covariates, covariance-based adjustment, adjusted treatment effect and uncertainty.
- Pre-treatment vs post-treatment variables; avoid adjustment using variables affected by the treatment.
- Revenue distribution diagnostics, outlier investigation, sensitivity analyses and transparent assumptions.
- Logging and explaining data-quality warnings in the dashboard.

Use in project: diagnostics page and original-vs-adjusted treatment-effect comparison.

Stage 5 — Core machine learning

- Train/validation/test splits, cross-validation, pipelines and preprocessing fitted only on training data.
- Categorical encoding, numerical scaling, imputation, class imbalance and feature leakage.
- Logistic Regression, Decision Tree, Random Forest and gradient boosting / XGBoost.
- Classification metrics: ROC-AUC, PR-AUC, precision, recall, F1, log loss and calibration.
- Regression metrics: MAE, RMSE and careful use of MAPE when values can be zero or near zero.
- Hyperparameter tuning, reproducible random seeds, baseline comparisons and error analysis.

Use in project: compare predictive models fairly and report actual held-out performance.

Stage 6 — Explainability and deep learning

- Feature importance and permutation importance; SHAP is an optional explainability extension.
- Neural-network basics: tensors, forward pass, loss, gradients, backpropagation and optimizers.
- MLP for tabular data, binary cross-entropy, ReLU, dropout, early stopping and validation monitoring.
- PyTorch or TensorFlow/Keras: choose one framework and use it consistently.
- Compare the MLP against Logistic Regression and XGBoost; accept the simpler model if it performs better.

Use in project: model comparison page and a clear account of why a model was selected.

Stage 7 — Causal inference and uplift modeling

- Potential outcomes, treatment assignment, average treatment effect (ATE) and individual treatment effect (ITE).
- Difference between predicting conversion and estimating the incremental effect of treatment.
- T-Learner: train separate outcome models for treatment and control, then compare predicted potential outcomes.
- X-Learner: understand its imputation and weighting idea; implement after T-Learner works.
- Uplift evaluation, calibration/uncertainty where feasible, segment stability and out-of-sample evaluation.
- Use randomized experiment data where possible; document assumptions and avoid causal claims from unsuitable observational data.

Use in project: uplift distribution, segment analysis and treatment targeting simulation.

Stage 8 — Business analytics and decision logic

- Conversion rate, absolute uplift, relative uplift, revenue per user and average order value.
- Translate estimated treatment effects into incremental conversions and revenue with uncertainty.
- Estimate implementation/marketing costs, contribution margin and ROI where inputs are available.
- Write transparent decision rules: deploy B, keep A, continue test, review cost-benefit or investigate data.
- What-if analysis for traffic volume, treatment effect, order value, costs and allocation.

Use in project: business impact engine and recommendation panel.

Stage 9 — Dashboard, APIs and deployment

- Streamlit: pages, widgets, forms, file upload, session state, charts, caching and error messages.
- Git/GitHub: commits, branches, .gitignore, README, issues and a clean repository.
- Testing with pytest; test statistical functions, data validation, model loading and decision rules.
- Save preprocessing and model pipelines together; pin dependencies in requirements.txt.
- Deployment on a suitable hosting platform; check current resource, privacy and sleep/availability limits.
- FastAPI basics, request/response schemas and API validation if a separate endpoint is useful.
- Optional Docker, MLflow tracking and PostgreSQL once the main application is stable.

Use in project: hosted demo, reproducible setup instructions and optional prediction API.

3. The Upgraded Technical Modules

3.1 Experiment data schema

Field	Type	Rules / meaning
user_id	String / ID	Stable analysis unit; decide how repeated sessions are aggregated.
variant	Category A/B	Randomized assignment; expected allocation should be known.
converted	0/1	Primary binary outcome.
revenue	Numeric	Revenue in a defined observation window; include zero values where appropriate.
pre_experiment_metric	Numeric	Historical activity measured before assignment; candidate CUPED covariate.
device / source / country	Category	Pre-treatment segment attributes, if available and appropriate.
sessions_before / previous_orders	Numeric	Historical user features; avoid post-treatment leakage.
experiment_time	Date/time	Used for duration, drift checks and time-aware validation.

Recommended data strategy

- Start with a reproducible synthetic dataset where the true treatment effect is known. This makes it possible to test whether the analysis recovers a known effect.
- Then use a suitable public dataset if one is available and its meaning is documented. Do not describe synthetic data as real company data.
- Keep the unit of randomization and analysis consistent. Repeated observations from the same user may require aggregation or user-clustered uncertainty estimates.
- Create a data dictionary and record the assignment ratio, primary metric, guardrails, analysis window and decision rule.

3.2 CUPED implementation concept

For outcome Y and pre-experiment covariate X, the common CUPED adjustment is $Y_{adj} = Y - \theta(X - \text{mean}(X))$, where θ is estimated from the covariance of X and Y divided by the variance of X. Estimate the treatment effect from adjusted outcomes, with an uncertainty estimator appropriate to the experiment design. Check that X is pre-treatment and correlated with Y. Report the effect and confidence interval both before and after adjustment.

3.3 SRM and sample-size checks

- SRM: specify the expected assignment probabilities, count users in each arm, calculate the chi-square statistic and p-value, and flag unexpected allocation.
- An SRM warning is a diagnostic, not proof of a specific implementation bug. Investigate logging, eligibility filters, assignment systems and exclusions.
- Power calculator: inputs should include baseline rate, MDE, alpha, desired power and allocation ratio. Clearly state the assumptions and whether the calculation is one- or two-sided.
- Do not repeatedly check ordinary p-values and stop whenever they become significant unless the design uses a valid sequential-testing procedure.

3.4 Uplift modeling

For each user, estimate $\mu_1(x) = E[Y \mid \text{treatment}, X=x]$ and $\mu_0(x) = E[Y \mid \text{control}, X=x]$. The T-Learner estimates uplift $\tau(x) = \mu_1(x) - \mu_0(x)$. For binary outcomes these are predicted conversion probabilities; for revenue they are expected outcomes on the chosen revenue scale. The difference is an estimate and needs evaluation, not a directly observed personal causal effect.

3.5 Business-impact formulas

Estimated incremental conversions ≈ eligible users × estimated absolute conversion-rate difference. Estimated incremental revenue should use a clearly defined revenue outcome or a justified conversion × order-value approximation. Estimated incremental profit = incremental contribution – implementation and campaign costs. Report uncertainty and assumptions; avoid treating model predictions as guaranteed gains.

4. Dashboard Specification

Make the interface look and behave like a small product analytics tool. Keep each page focused and show warnings when the data cannot support a trustworthy conclusion.

Page	Key contents	User action
Overview	Sample sizes, conversion rates, absolute/relative uplift, confidence interval and status	Understand the experiment at a glance
Data Quality	Missingness, duplicates, assignment balance, SRM warning, leakage checks	Investigate before interpreting results
Statistical Analysis	Hypothesis test, p-value, confidence intervals, effect size, guardrails	Assess evidence and practical importance
CUPED	Covariate selection, original vs adjusted effect and interval, variance reduction	Compare precision transparently
Power Planner	Baseline rate, MDE, alpha, power and allocation ratio; required sample size	Plan a future experiment
ML Predictions	Model comparison, ROC-AUC/PR-AUC, log loss, calibration and feature importance	Review predictive performance
Uplift & Segments	Estimated uplift distribution, segment effects, uncertainty and caveats	Explore who may benefit from B
Business Impact	Incremental conversions, revenue/profit estimate, cost and ROI	Translate effects into business terms
What-if Simulator	Traffic, treatment effect, order value, cost and allocation inputs	Explore assumptions—not guarantees
Model Card	Dataset, target, split strategy, metrics, limitations, versions and intended use	Understand reliability and boundaries

Recommended dashboard behavior

- Never show example figures as if they came from the uploaded dataset. Label simulated/demo values clearly.
- Show effect sizes and confidence intervals alongside p-values.
- Show an explicit warning when SRM or data-quality checks fail.
- Explain whether a result is predictive, descriptive, or causal.
- Keep the primary metric and guardrail metrics visible, and make the recommendation logic inspectable.
- Allow users to upload a CSV with schema validation and a clear error message if required columns are missing.

5. Recommended Repository Structure

Keep analysis logic outside the Streamlit UI so functions can be tested and reused in notebooks, scripts and optional API endpoints.

```
ExperimentIQ/ ■■■ data/ ■ ■■■ raw/ # ignored if private or large ■ ■■■ processed/ ■ ■■■ synthetic/ ■■■ notebooks/ ■ ■■■ 01_data_generation.ipynb ■ ■■■ 02_eda_validation.ipynb ■ ■■■ 03_ab_statistics.ipynb ■ ■■■ 04_cuped_power_srm.ipynb ■ ■■■ 05_ml_models.ipynb ■ ■■■ 06_mlp_comparison.ipynb ■ ■■■ 07_uplift_modeling.ipynb ■■■ src/experimentiq/ ■ ■■■ data/ # cleaning.py, validation.py ■ ■■■ statistics/ # ab_tests.py, cuped.py, power.py, srm.py ■ ■■■ features/ # engineering.py, preprocessing.py ■ ■■■ models/ # train_ml.py, train_mlp.py, evaluate.py ■ ■■■ uplift/ # t_learner.py, x_learner.py, evaluation.py ■ ■■■ business/ # impact.py, recommendations.py ■ ■■■ utils/ # config.py, logging.py ■■■ dashboard/ ■ ■■■ app.py ■ ■■■ pages/ ■■■ api/ # optional FastAPI service ■■■ models/ # model artifacts if small and appropriate ■■■ tests/ ■■■ requirements.txt ■■■ README.md ■■■ MODEL_CARD.md ■■■ Dockerfile # optional ■■■ .gitignore
```

Suggested tools and libraries

Area	Suggested tools	Learn first
Core language/data	Python, NumPy, Pandas	Python + Pandas
Statistics	SciPy, statsmodels	SciPy / statsmodels basics
ML	scikit-learn, XGBoost	scikit-learn pipelines
Deep learning	PyTorch or TensorFlow/Keras	One framework only
Causal ML	scikit-learn first; optional EconML or CausalML	T-Learner concepts
Visualization	Plotly or Matplotlib	One charting approach
Dashboard	Streamlit	Multi-page dashboard
Testing / engineering	pytest, Git, logging	Tests and version control
Optional deployment	FastAPI, Docker, MLflow, PostgreSQL	Add only after core works

6. Suggested 6-Week Build Plan

This is a practical target for a student who can work on the project consistently. Extend the schedule if you are learning statistics or ML from scratch.

Week	Learn / implement	Deliverable / acceptance check
1	Python/Pandas refresh, data schema, synthetic data, EDA and data validation	Reproducible dataset generator; data dictionary; no silent schema errors
2	A/B statistics, confidence intervals, effect sizes, SRM and sample-size planning	Tested statistical functions; known-effect synthetic tests
3	CUPED, revenue skew diagnostics and bootstrap/sensitivity analysis	Original vs adjusted effects; documented assumptions and warnings
4	Logistic Regression, Random Forest, XGBoost, evaluation and calibration	Held-out model comparison with no leakage
5	MLP comparison, T-Learner uplift and segment analysis	Baseline uplift pipeline; documented limitations
6	Business-impact logic, Streamlit pages, tests, README and deployment	Working demo, setup instructions, screenshots and model card

If time is limited, prioritize in this order

- First: trustworthy A/B analysis, data validation, SRM and sample-size planning.
- Second: CUPED and the business-impact dashboard.
- Third: strong ML baselines and honest evaluation.
- Fourth: T-Learner uplift modeling.
- Fifth: MLP, X-Learner, FastAPI, Docker, MLflow and PostgreSQL as extensions.

Definition of done

- A fresh clone can install dependencies and run the application using the README.
- Statistical functions have tests, including a synthetic dataset with a known treatment effect.
- Train/test separation and pre-treatment feature rules are documented.
- The dashboard shows uncertainty, warnings, and assumptions—not just a winner label.
- Model scores and business-impact figures are measured from reproducible runs.
- The hosted demo and GitHub README link to each other; no secrets or private data are committed.

7. Evaluation Checklist and Common Mistakes

Use this as the final review checklist before publishing the project or discussing it in interviews.

Data and experimental design

- The randomization unit and analysis unit are defined.
- Assignment counts and expected allocation are checked; SRM warnings are visible.
- Missing values, duplicates, exclusions and outliers are documented.
- Features used for CUPED or ML are available before treatment or otherwise justified.
- Primary metric, guardrails, analysis window and decision rule are defined before looking at results.

Statistics and causal claims

- Report effect size and confidence interval, not only a p-value.
- Distinguish statistical significance from business significance.
- Do not claim CUPED always changes the estimated effect or always improves precision; verify it on the data.
- Do not use naive repeated significance checks to stop an experiment early.
- Uplift is estimated from potential outcomes; individual treatment effects are not directly observed.
- Do not infer causality from observational data without a defensible identification strategy.

Machine learning and engineering

- Fit preprocessing only on training data and keep the test set untouched until final evaluation.
- Do not predict the A/B winner using only the variant column.
- Use appropriate metrics for imbalanced conversion data; accuracy alone is inadequate.
- Compare the MLP fairly against simpler baselines; do not force deep learning to win.
- Test data validation, statistical calculations, model loading and recommendation rules.
- Record package versions, model artifacts and experiment configuration.

Business and communication

- Label synthetic data and illustrative numbers clearly.
- State the population, assumptions, uncertainty and limitations of every business-impact estimate.
- Do not promise ROI or claim company-wide gains from a simulated experiment.
- Use only measured metrics in the resume and project README.

8. Resume Positioning and Interview Preparation

Possible project description — update it only after you have implemented and measured each feature:

ExperimentIQ | ML-Powered A/B Testing & Business Experiment Intelligence
Built an experimentation analytics platform combining hypothesis testing, confidence intervals, CUPED variance reduction, sample-ratio diagnostics and power planning. Compared predictive models for conversion outcomes and implemented uplift modeling to estimate treatment-effect heterogeneity. Developed a Streamlit dashboard to inspect experiment quality, segment performance and estimated business impact.

Do not add performance numbers until you have measured them on a documented held-out dataset. Do not list X-Learner, FastAPI, Docker or any other feature as completed until it actually works.

Questions you should be able to answer

- Why is A/B testing needed if you already have a predictive ML model?
- What does a p-value tell you, and what does it not tell you?
- What is the difference between absolute uplift and relative uplift?
- How does CUPED reduce variance, and why must the covariate be pre-treatment?
- What does SRM mean, and why can it invalidate an experiment analysis?
- How do power, alpha, MDE and sample size relate to one another?
- How does a T-Learner differ from an ordinary conversion classifier?
- Why can a simpler model outperform an MLP on tabular data?
- How do you prevent leakage and make sure test metrics are honest?
- How do you translate an estimated treatment effect into revenue while showing uncertainty?

9. Recommended Learning Resources

Use official documentation as the reference when implementing APIs; pair it with one structured course or tutorial per topic. Resource pages and package APIs can change, so verify the current version before installation.

Topic	Resource
Python	Python official tutorial — docs.python.org/3/tutorial/
Pandas	Pandas getting started — pandas.pydata.org/docs/getting_started/
NumPy	NumPy quickstart — numpy.org/doc/stable/user/quickstart.html
Statistics	SciPy statistics — docs.scipy.org/doc/scipy/tutorial/stats.html
Statistical models	statsmodels documentation — www.statsmodels.org/
Machine learning	scikit-learn user guide — scikit-learn.org/stable/user_guide.html
XGBoost	XGBoost documentation — xgboost.readthedocs.io/
Deep learning	PyTorch tutorials — pytorch.org/tutorials/
Causal ML	EconML documentation — econml.azurewebsites.net/; CausalML documentation — econml? verify package site before use
Streamlit	Streamlit documentation — docs.streamlit.io/
FastAPI	FastAPI tutorial — fastapi.tiangolo.com/tutorial/
Git	Pro Git book — git-scm.com/book/en/v2
pytest	pytest documentation — docs.pytest.org/

Note: choose either PyTorch or TensorFlow/Keras; either EconML or CausalML as an optional uplift library. Do not attempt to learn every tool simultaneously.

10. Final Build Order — One-Page Checklist

1. Define the experiment, unit of analysis, primary metric, guardrails and expected allocation.
2. Generate or select a documented dataset and build data validation.
3. Perform EDA and check assignment balance / SRM.
4. Implement baseline A/B statistics, confidence intervals and effect sizes.
5. Add power and sample-size planning.
6. Add CUPED with a valid pre-experiment covariate and compare precision.
7. Build ML pipelines: Logistic Regression → Random Forest → XGBoost.
8. Evaluate classification calibration and, if needed, revenue regression.
9. Build and compare an MLP; report results honestly.
10. Implement T-Learner uplift modeling; add X-Learner only after validation.
11. Build business-impact calculations and transparent recommendation rules.
12. Create Streamlit pages for overview, diagnostics, CUPED, power, ML, uplift and business impact.
13. Add unit tests, README, model card, architecture diagram and screenshots.
14. Deploy the dashboard and link it from GitHub.
15. Add optional FastAPI, Docker, MLflow or PostgreSQL only when they serve a clear purpose.

Final recommendation

The strongest version of ExperimentIQ is a carefully validated product: statistics first, ML second, uplift modeling third, and deployment last. Prioritize correctness, reproducibility, and clear business explanations over adding more technologies.

End of blueprint